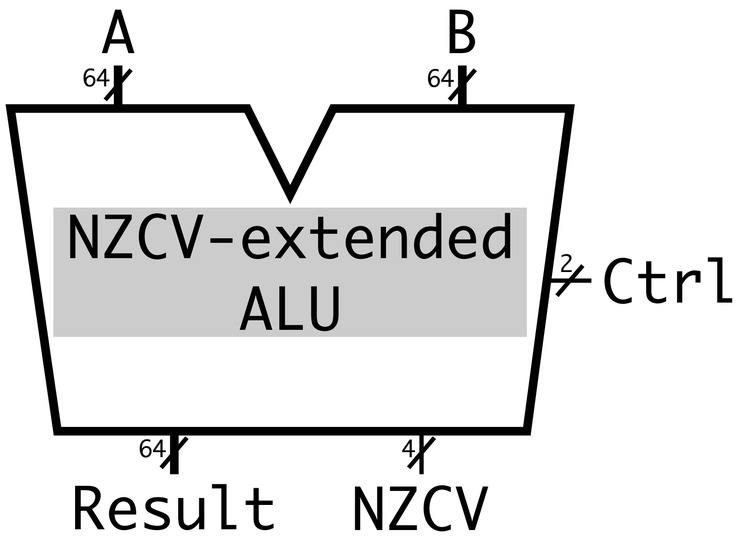


Arithmetic Logic Unit

Alex Breuer

Overview and Schedule

- Arithmetic Logic Unit (ALU) is a key component of a processor's data path (later in the course)
- Performs different logical or arithmetic operations
- ALU might output additional information (flags) about the result of the performed operation
- Our ALU will support the following:
  - Operations: add, sub, bitwise and, bitwise or
  - Output of NZCV flags: Negative, Zero, Carry, oVerflow
- Roadmap:
  - Have a look at pseudocode of A64 instructions to understand how our ALU is supposed to work
  - Implement an ALU with the desired features and study its behavior



NZCV Flags

- Many A64 instructions write condition flags (also called NZCV flags)
- Flags provide information on the result of such an instruction
- Used extensively for conditional branching which modifies the program counter based on these flags
- We'll cover the details of branching in a few weeks when discussing AArch64
- Now: Look at the pseudocode for ANDS, ADDS and SUBS, which set condition flags

Name, Location	Meaning
N (Negative), bit [31]	Negative condition flag. Set to 1 if the result of the last flag-setting instruction was negative.
Z (Zero), bit [30]	Zero condition flag. Set to 1 if the result of the last flag-setting instruction was zero, and to 0 otherwise. A result of zero often indicates an equal result from a comparison.
C (Carry), bit [29]	Carry condition flag. Set to 1 if the last flag-setting instruction resulted in a carry condition, for example an unsigned overflow on an addition.
V (oVerflow), bit [28]	Overflow condition flag. Set to 1 if the last flag-setting instruction resulted in an overflow condition, for example a signed overflow on an addition.

Source: https://developer.arm.com/documentation/ddi0595/2021-03/AArch64-Registers/NZCV--Condition-Flags?lang=en

Targeted Behavior: Pseudocode

ANDS

```
constant bits(datasize) operand1 = X[n, datasize];
constant bits(datasize) operand2 = ShiftReg(m, shift_type,
    shift_amount, datasize);

constant bits(datasize) result = operand1 AND operand2;
X[d, datasize] = result;
PSTATE.<N,Z,C,V> = result<datasize-1>:IsZeroBit(result):'00';
```

Source: Arm pseudocode

Bitwise AND produces the result. The most significant result bit sets N; a zero result sets Z. C and V are cleared.

ANDS: Example 1

Four-bit example: `datasize = 4`.

```
constant bits(datasize) operand1 = X[n, datasize];              1010
constant bits(datasize) operand2 = ShiftReg(m, shift_type,      1011
    shift_amount, datasize);

constant bits(datasize) result = operand1 AND operand2;         1010
X[d, datasize] = result;
PSTATE.<N,Z,C,V> = result<datasize-1>:IsZeroBit(result):'00'; NZCV = 1000
```

Source: Arm pseudocode

Compute the bitwise AND, then derive N and Z from the result. C and V are cleared to zero.

ANDS: Example 2

Four-bit example: `datasize = 4`.

```
constant bits(datasize) operand1 = X[n, datasize];             0101
constant bits(datasize) operand2 = ShiftReg(m, shift_type,     1010
    shift_amount, datasize);

constant bits(datasize) result = operand1 AND operand2;        0000
X[d, datasize] = result;
PSTATE.<N,Z,C,V> = result<datasize-1>:IsZeroBit(result):'00'; NZCV = 0100
```

Source: Arm pseudocode

Compute the bitwise AND, then derive N and Z from the result. C and V are cleared to zero.

ADDS / SUBS

ADDS Use the shifted second operand without complementing it; carry input is `0`.

```
constant bits(datasize) operand1 = X[n, datasize];
constant bits(datasize) operand2 =
    ShiftReg(m, shift_type,shift_amount, datasize);
bits(datasize) result;
bits(4) nzcv;
(result, nzcv) = AddWithCarry(operand1, operand2, '0');
X[d, datasize] = result;
PSTATE.<N,Z,C,V> = nzcv;
```

Source: Arm pseudocode

SUBS Complement the shifted second operand; carry input is 1.

```
constant bits(datasize) operand1 = X[n, datasize];
constant bits(datasize) operand2 =
    NOT(ShiftReg(m, shift_type, shift_amount, datasize));
bits(datasize) result;
bits(4) nzcv;
(result, nzcv) = AddWithCarry(operand1, operand2, '1');
X[d, datasize] = result;
PSTATE.<N,Z,C,V> = nzcv;
```

Source: Arm pseudocode

AddWithCarry

Add two N-bit operands and a carry input.

Return the result and all four condition flags.

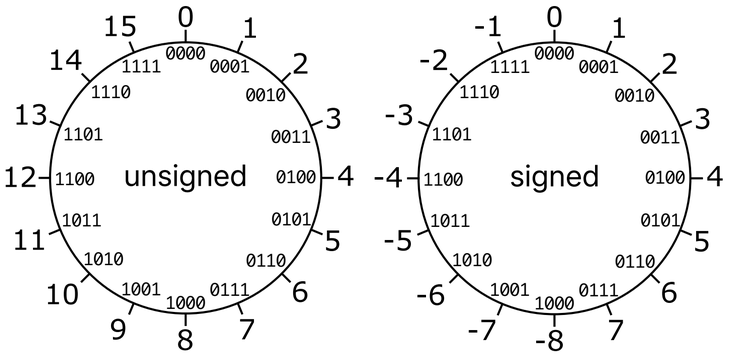
```
(bits(N), bits(4)) AddWithCarry(bits(N) x, bits(N) y, bit carry_in)
    constant integer unsigned_sum = UInt(x) + UInt(y) + UInt(carry_in);
    constant integer signed_sum = SInt(x) + SInt(y) + UInt(carry_in);
    // Low N bits also equal signed_sum<N-1:0>.
    constant bits(N) result = unsigned_sum<N-1:0>;
    constant bit n = result<N-1>;
    constant bit z = if IsZero(result) then '1' else '0';
    constant bit c = if UInt(result) == unsigned_sum then '0' else '1';
    constant bit v = if SInt(result) == signed_sum then '0' else '1';
    return (result, n:z:c:v);
```

Source: Arm pseudocode

Width N counts result bits; `n:z:c:v` becomes PSTATE.NZCV.

C is 1 when `UInt(result)` differs from `unsigned_sum`.

V is 1 when `SInt(result)` differs from `signed_sum`.




```
integer UInt(bits(N) x)
    result = 0;
    for i = 0 to N-1
        if x<i> == '1' then
            result = result + 2^i;
    return result;
```

```
integer SInt(bits(N) x)
    result = 0;
    for i = 0 to N-1
        if x<i> == '1' then
            result = result + 2^i;
    if x<N-1> == '1' then
        result = result - 2^N;
    return result;
```

ADDS: Example 1

ADDS Use the shifted second operand without complementing it; carry input is 0.

```
constant bits(datasize) operand1 = X[n, datasize];      0101
constant bits(datasize) operand2 =
    ShiftReg(m, shift_type,shift_amount, datasize);     0110
bits(datasize) result;
bits(4) nzcv;
(result, nzcv) = AddWithCarry(operand1, operand2, '0'); x=0101, y=0110, carry_in=0
X[d, datasize] = result;
PSTATE.<N,Z,C,V> = nzcv;
```

Source: Arm pseudocode

SUBS Complement the shifted second operand; carry input is 1.

```
constant bits(datasize) operand1 = X[n, datasize];
constant bits(datasize) operand2 =
    NOT(ShiftReg(m, shift_type, shift_amount, datasize));
bits(datasize) result;
bits(4) nzcv;
(result, nzcv) = AddWithCarry(operand1, operand2, '1');
X[d, datasize] = result;
PSTATE.<N,Z,C,V> = nzcv;
```

Source: Arm pseudocode

AddWithCarry: Example 1

Four-bit example: width `N = 4`.

`x=0101, y=0110, carry_in=0`

```
(bits(N), bits(4)) AddWithCarry(bits(N) x, bits(N) y, bit carry_in)
    constant integer unsigned_sum = UInt(x) + UInt(y) + UInt(carry_in); 11
    constant integer signed_sum = SInt(x) + SInt(y) + UInt(carry_in);   11
    // Low N bits also equal signed_sum<N-1:0>.
    constant bits(N) result = unsigned_sum<N-1:0>;                      1011
    constant bit n = result<N-1>;                                       1
    constant bit z = if IsZero(result) then '1' else '0';               0
    constant bit c = if UInt(result) == unsigned_sum then '0' else '1'; 0
    constant bit v = if SInt(result) == signed_sum then '0' else '1';   1
    return (result, n:z:c:v);
```

Source: Arm pseudocode

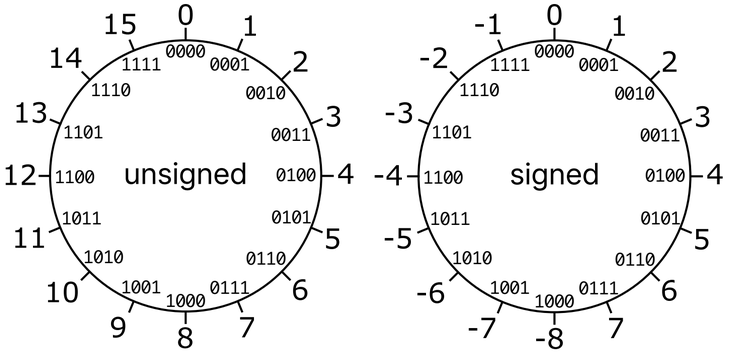
Width N counts result bits; `n:z:c:v` becomes PSTATE.NZCV.

C is 1 when `UInt(result)` differs from `unsigned_sum`.

V is 1 when `SInt(result)` differs from `signed_sum`.

`UInt('1011') = 11`, but `SInt('1011') = -5`.

The signed sum 11 does not fit in four bits: **NZCV = 1001**.




```
integer UInt(bits(N) x)
    result = 0;
    for i = 0 to N-1
        if x<i> == '1' then
            result = result + 2^i;
    return result;
```

```
integer SInt(bits(N) x)
    result = 0;
    for i = 0 to N-1
        if x<i> == '1' then
            result = result + 2^i;
    if x<N-1> == '1' then
        result = result - 2^N;
    return result;
```

SUBS: Example 2

ADDS Use the shifted second operand without complementing it; carry input is 0.

```
constant bits(datasize) operand1 = X[n, datasize];
constant bits(datasize) operand2 =
    ShiftReg(m, shift_type,shift_amount, datasize);
bits(datasize) result;
bits(4) nzcv;
(result, nzcv) = AddWithCarry(operand1, operand2, '0');
X[d, datasize] = result;
PSTATE.<N,Z,C,V> = nzcv;
```

Source: Arm pseudocode

SUBS Complement the shifted second operand; carry input is 1.

```
constant bits(datasize) operand1 = X[n, datasize];          0101
constant bits(datasize) operand2 =
    NOT(ShiftReg(m, shift_type, shift_amount, datasize)); NOT(0011) = 1100
bits(datasize) result;
bits(4) nzcv;
(result, nzcv) = AddWithCarry(operand1, operand2, '1');   x=0101, y=1100, carry_in=1
X[d, datasize] = result;
PSTATE.<N,Z,C,V> = nzcv;
```

Source: Arm pseudocode

AddWithCarry: Example 2

Four-bit example: width `N = 4`.

`x=0101, y=1100, carry_in=1`

```
(bits(N), bits(4)) AddWithCarry(bits(N) x, bits(N) y, bit carry_in)
    constant integer unsigned_sum = UInt(x) + UInt(y) + UInt(carry_in); 18
    constant integer signed_sum = SInt(x) + SInt(y) + UInt(carry_in);   2
    // Low N bits also equal signed_sum<N-1:0>.
    constant bits(N) result = unsigned_sum<N-1:0>;                      0010
    constant bit n = result<N-1>;                                       0
    constant bit z = if IsZero(result) then '1' else '0';               0
    constant bit c = if UInt(result) == unsigned_sum then '0' else '1'; 1
    constant bit v = if SInt(result) == signed_sum then '0' else '1';   0
    return (result, n:z:c:v);
```

Source: Arm pseudocode

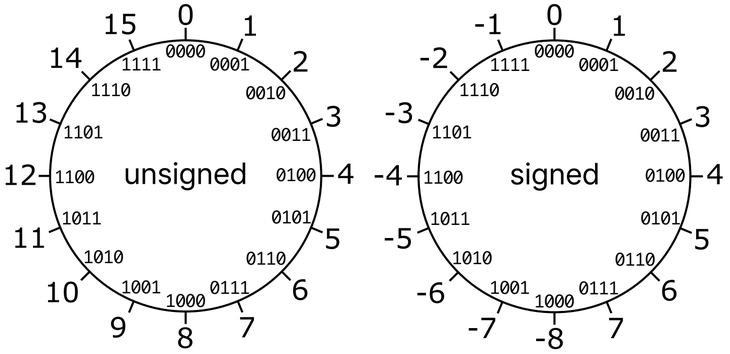
Width `N` counts result bits; `n:z:c:v` becomes PSTATE.NZCV.

C is 1 when `UInt(result)` differs from `unsigned_sum`.

V is 1 when `SInt(result)` differs from `signed_sum`.

`UInt('0010') = 2` differs from the unsigned sum 18.

`SInt('0010') = 2` equals the signed sum: **NZCV = 0010**.




```
integer UInt(bits(N) x)
    result = 0;
    for i = 0 to N-1
        if x<i> == '1' then
            result = result + 2^i;
    return result;
```

```
integer SInt(bits(N) x)
    result = 0;
    for i = 0 to N-1
        if x<i> == '1' then
            result = result + 2^i;
    if x<N-1> == '1' then
        result = result - 2^N;
    return result;
```

Implementation

Arithmetic Logic Unit without Flags

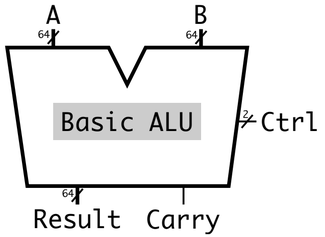
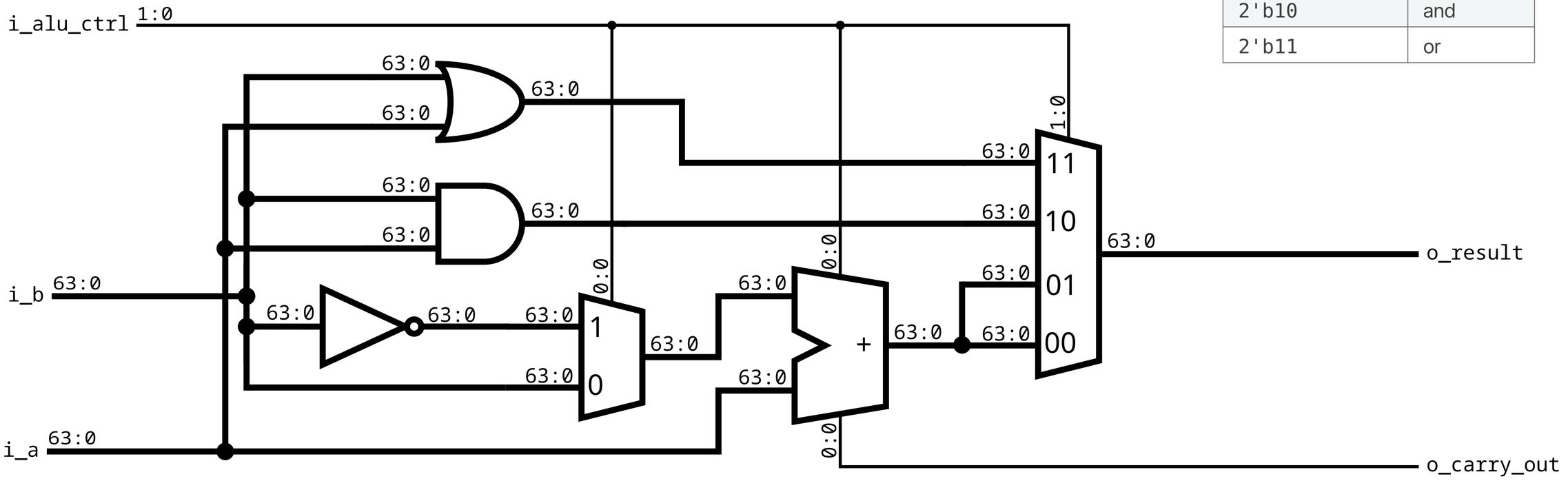


Table 7.1 Functions supported by our ALU depending on a two-bit control signal.

Control Signal	Function
2'b00	add
2'b01	sub
2'b10	and
2'b11	or


i_alu_ctrl
1:0
63:0
63:0
63

NZCV: Negative and Zero

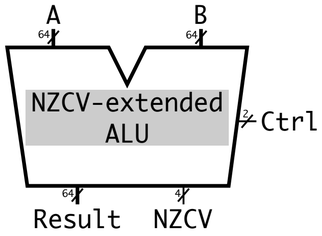
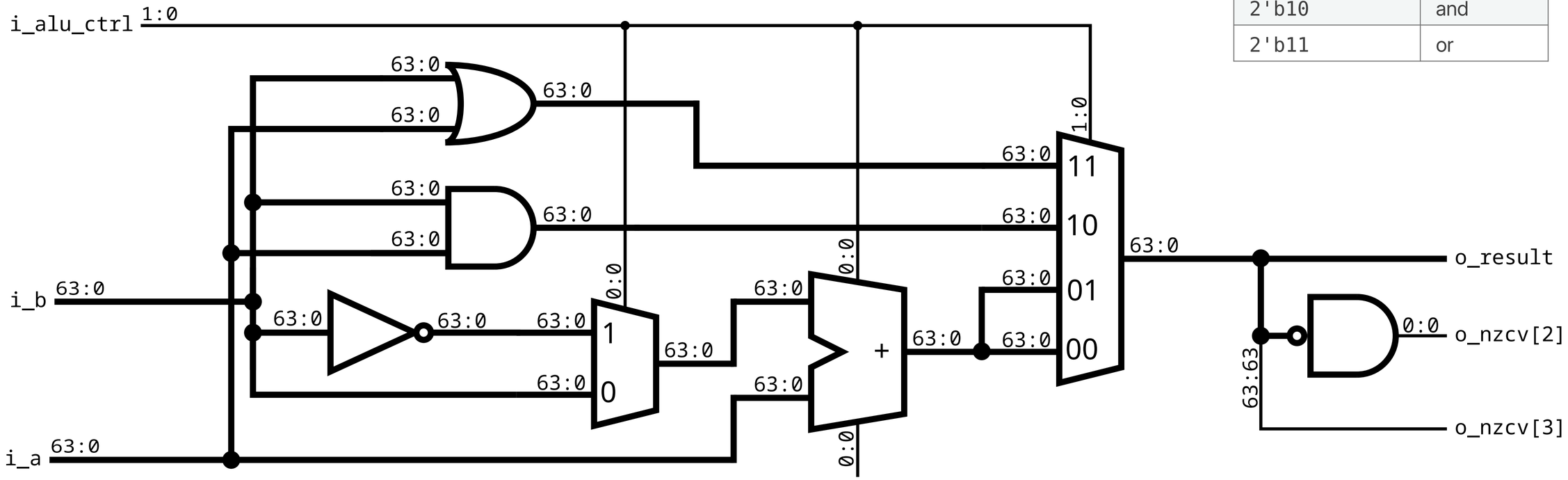


Table 7.1 Functions supported by our ALU depending on a two-bit control signal.

Control Signal	Function
2'b00	add
2'b01	sub
2'b10	and
2'b11	or


i_alu_ctrl
1:0
i_b
63:0
i_a
63:0
63:0
63:0

NZCV: Negative, Zero and Carry

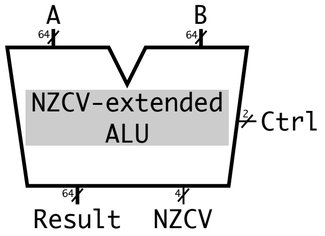
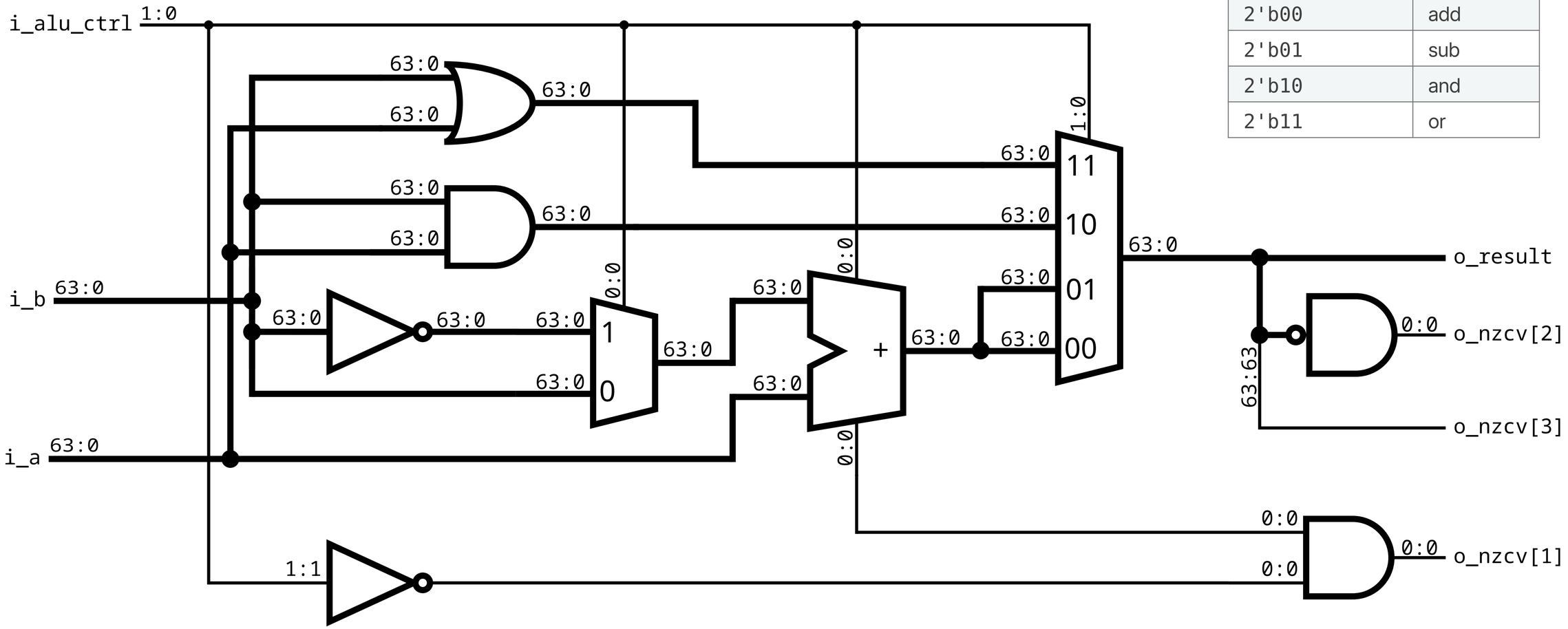


Table 7.1 Functions supported by our ALU depending on a two-bit control signal.

Control Signal	Function
2'b00	add
2'b01	sub
2'b10	and
2'b11	or



NZCV: Negative, Zero, Carry and oVerflow

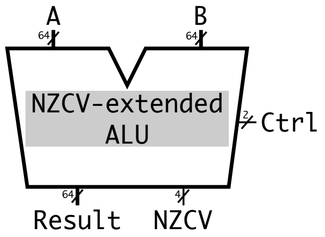
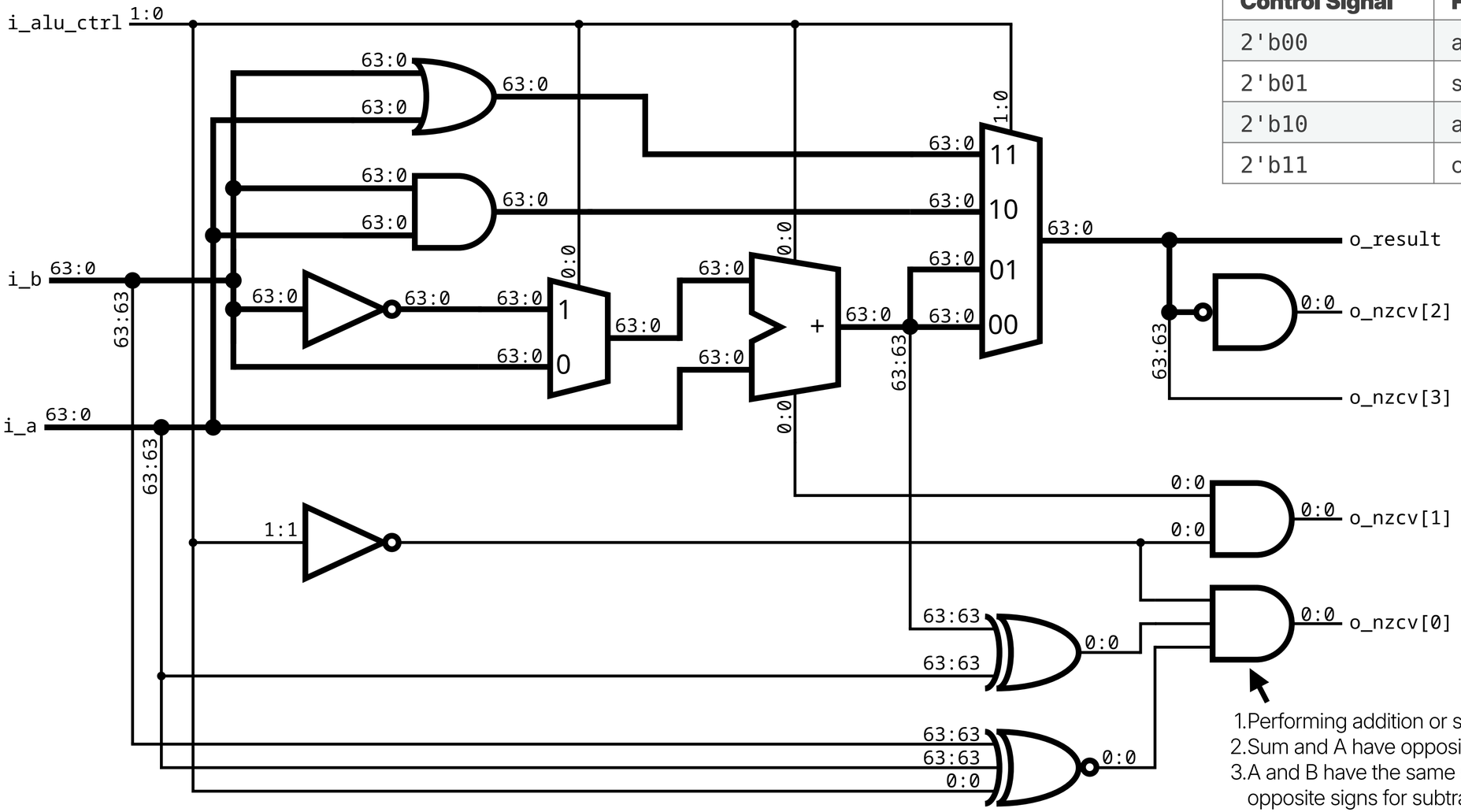


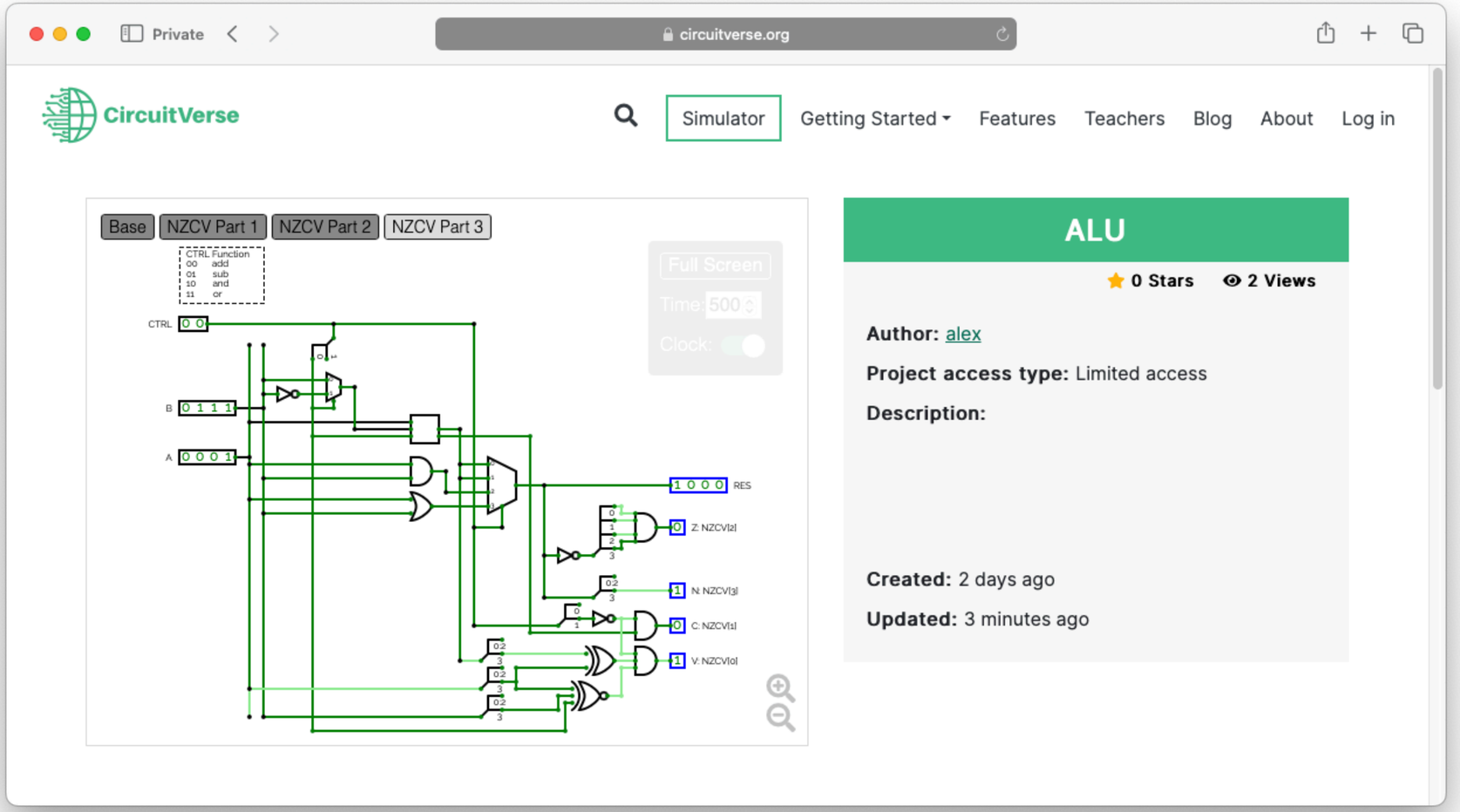
Table 7.1 Functions supported by our ALU depending on a two-bit control signal.

Control Signal	Function
2'b00	add
2'b01	sub
2'b10	and
2'b11	or



1. Performing addition or subtraction?
2. Sum and A have opposite signs?
3. A and B have the same sign for addition, opposite signs for subtraction?

Simulation



https://circuitverse.org/users/151679/projects/alu-2cb3c55a-9666-4c9a-b5b6-603a2dc40354

Summary and Outlook

- Studied pseudocode of A64 instructions to understand how our ALU should behave
- Outputting NZCV flags requires additional logic; oVerflow flag is more challenging than the others
- Roadmap:
  - Sequential logic
  - AArch64 and respective instructions; incl. instructions relying on NZCV flags
  - Control and data path of a processor; ALU is a key component of the data path

